Then we can see that

$$\mathsf{GNSA}(\tilde{X}, \tilde{Y}) = \frac{1}{s^2} \sum_{1 \leq i,j \leq N} \left| \frac{d(x_i, x_j)}{D_X} - \frac{d(y_i, y_j)}{D_Y} \right| \mathbb{I}_{\tilde{X}}(x_i) \, \mathbb{I}_{\tilde{X}}(x_j) \, .$$

Hence, taking expectation over $\tilde{X}, \tilde{Y}$, we get

$$\mathop{\mathbb{E}}_{\tilde{X},\tilde{Y}} \left[\mathsf{GNSA}(\tilde{X}, \tilde{Y}) \right] = \frac{1}{s^2} \mathop{\mathbb{E}}_{\tilde{X},\tilde{Y}} \left[\sum_{1 \leq i,j \leq N} \left| \frac{d(x_i, x_j)}{D_X} - \frac{d(y_i, y_j)}{D_Y} \right| \mathbb{I}_{\tilde{X}}(x_i) \, \mathbb{I}_{\tilde{X}}(x_j) \right] .$$

By linearity of expectation, we get

$$\mathop{\mathbb{E}}_{\tilde{X},\tilde{Y}} \left[\mathsf{GNSA}(\tilde{X}, \tilde{Y}) \right] = \frac{1}{s^2} \sum_{1 \leq i,j \leq N} \left| \frac{d(x_i, x_j)}{D_X} - \frac{d(y_i, y_j)}{D_Y} \right| \mathop{\mathbb{E}}_{\tilde{X},\tilde{Y}} \left[\mathbb{I}_{\tilde{X}}(x_i) \, \mathbb{I}_{\tilde{X}}(x_j) \right] .$$

Assuming $s >> 1$, each x_i is independently in $\tilde{X}$ with probability s/N, hence, we get

$$\mathop{\mathbb{E}}_{\tilde{X},\tilde{Y}} \left[\mathsf{GNSA}(\tilde{X}, \tilde{Y}) \right] = \frac{1}{s^2} \sum_{1 \leq i,j \leq N} \left| \frac{d(x_i, x_j)}{D_X} - \frac{d(y_i, y_j)}{D_Y} \right| \frac{s}{N} \frac{s}{N} .$$

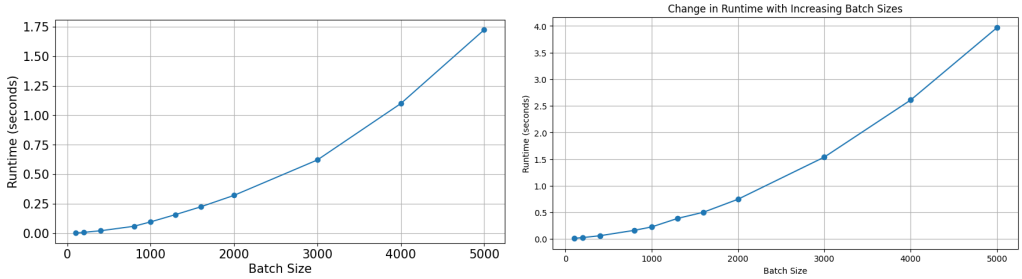
Hence, we have

$$\mathop{\mathbb{E}}_{\tilde{X},\tilde{Y}} \left[\mathsf{GNSA}(\tilde{X}, \tilde{Y}) \right] = \mathsf{GNSA}(X, Y).$$

□

H NSA running time over increasing batch sizes

Theoretically, NSA is quadratic in the number of points as we need to calculate pairwise distances. But since we use torch functions and the data is loaded on the GPU, empirical running times are significantly lower. Figure 5(a) presents the running time of GlobalNSA and Figure 5(a) presents the running time of NSA (Global and Local combined) on two embedding spaces of very high dimensionality (200,000). We notice that even when we get to a very high batch size (5000), the running time scaling is nearly linear. It is worth noting that these running times are higher as the dimensionality is very high, in most practical scenarios, your embeddings would lie in a lower dimension.

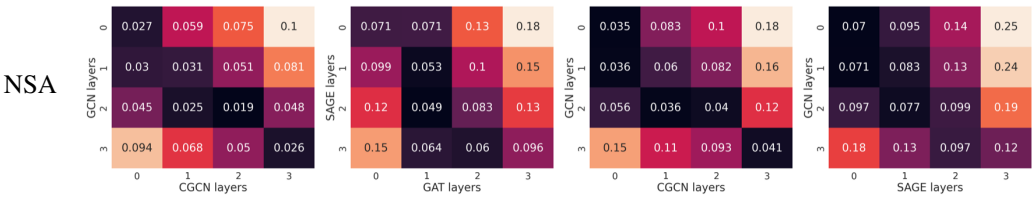


(a) (b)

Figure 5: Running time of NSA over increasing batch sizes. (a) Running time of GlobalNSA over increasing batch size. (b) Running time of NSA over increasing batch size

I Are GNNs Task Specific Learners?

We use NSA to investigate whether representations produced by GNNs are specific to the downstream task. For this, we explored the similarity of representations of different GNN architectures when trained on the same task versus their similarity when trained on different downstream tasks.



(a) GCN vs CGCN (b) GraphSAGE vs GAT (c) GCN vs CGCN (d) GCN vs GSAGE

Figure 6: Cross Architecture Tests using NSA on the Amazon Computers Dataset. (a) Layerwise NSA values between GCN and ClusterGCN on Node Classification (b) Layerwise NSA values between GraphSAGE and GAT on Node Classification (c) Layerwise NSA values between GCN and ClusterGCN on Link Prediction (d) Layerwise NSA values between GCN and GSAGE on Link Prediction. Similar architectures showcase a layerwise pattern when trained on the same task.

I.1 Cross Architecture Tests on the Same Downstream Task

We tested layerwise representational similarity on the task of node classification (on the Amazon Computers Dataset) across different GNN architectures: GCN, ClusterGCN (CGCN), GraphSAGE, and GAT. Just like in the sanity tests, models that are similar in architecture or training paradigms have a higher layerwise similarity: this implies that the highest similarity is between GCN and ClusterGCN that differ only in their training paradigms. We also observe (a less pronounced) linear relationship between the other pairs of architectures. These results are shown in Figure 6.

I.2 Architecture Tests on Different Downstream Tasks

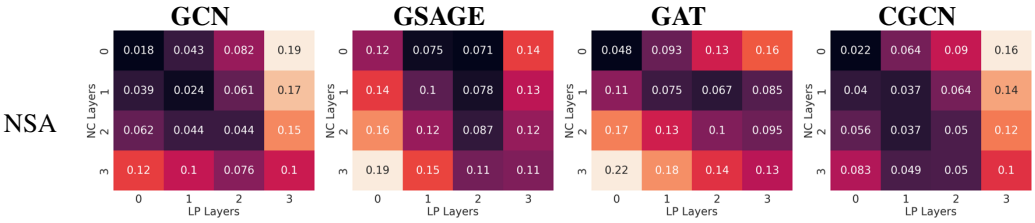


Figure 7: Effect of Downstream Task on Representations Achieved by the Same Architecture (Amazon Computers Dataset). The layerwise dissimilarity of four GNN architectures is compared on two different downstream tasks: Node Classification and Link Prediction. There is no observable correlation across layers suggesting that the GNNs generate different representation spaces for the same dataset on different downstream tasks.

We conducted experiments with different downstream tasks to assess layerwise similarity between two models, both of which shared structural identity except for disparities in their final layers. Specifically, we employed node classification and link prediction as the two downstream tasks for our investigations. We present our findings in Figure 7. They reveal that there is little relationship across layers for any of the four architectures.

The results presented in this section indicate that that different GNN architectures have similar representation spaces when trained on the same downstream task and conversely, similar architectures have different representation spaces when trained on different downstream tasks. Extending this idea further, it should be possible to train Graph Neural Networks to conform to a task specific representation template. This structural template will be agnostic of GNN architectures and provide a high degree of functional similarity. If we train GNNs to minimize discrepancy loss with such a task specific template, we could train more directly and without adding downstream layers of tasks.

J Downstream Task Analysis with the STS Dataset

We assess the performance of embeddings derived from NSA-AE in the context of the Semantic Text Similarity (STS) Task. We employ Google's Word2Vec word vectors as the initial dataset, processing

these 300-dimensional vectors through both a standard autoencoder and the NSA-Autoencoder to achieve reduced dimensions of 128 and 64. The efficacy of these dimensionality-reduced embeddings is then evaluated using the STS Multi EN dataset (Cer et al., 2017). This evaluation aims to determine if the word vectors, once dimensionally reduced, maintain their alignment, as evidenced by a strong positive correlation in similarity scores on the STS dataset. The comparative results, as detailed in Table 3, demonstrate that the NSA-Autoencoder outperforms the traditional autoencoder in preserving semantic accuracy at both 128 and 64 dimensions.

Table 3: Pearson Correlation between the similarities output from the original embeddings and the similarities output from the reduced embeddings. NSA-AE shows a much higher correlation with the similarity outputs obtained by evaluating the original word2vec embeddings on the STS multi EN dataset

Latent Dimension	**Basic AutoEncoder**	**NSA AutoEncoder**
128	-0.518	0.7632
64	0.567	0.7619

K Running Time and Reconstruction Loss for Autoencoder

We show that using NSA along with Mean Squared Error in an autoencoder does not significantly compromise the model's ability to reconstruct the original embedding. Additionally, NSA-AE is more efficient than RTD-AE and Topo-AE. The results are presented in Table 4

Table 4: Reconstruction Loss and Time Per Epoch for different Autoencoder architectures. All architectures were trained for 250 epochs with the auxiliary loss (TopoLoss, RTD, NSA) kicking in after 60 epochs.

		Metric		
Dataset	Method	Time per epoch	Train MSE	Test MSE
MNIST	AE	2.344	7.64e-3	8.62e-3
	TopoAE	39.168	6.89e-3	8.16e-3
	RTD-AE	51.608	9.36e-3	1.07e-2
	NSA-AE	5.816	8.75e-3	1.00e-2
F-MNIST	AE	6.436	8.99e-03	9.79e-03
	TopoAE	37.26	8.94-03	9.83e-03
	RTD-AE	59.94	1.12e-02	1.24e-02
	NSA-AE	5.764	9.49e-03	1.04e-02
CIFAR-10	AE	5.16	1.56e-02	1.65e-02
	TopoAE	58.664	1.54e-02	1.68e-02
	RTD-AE	56.172	1.60e-02	1.91e-02
	NSA-AE	6.996	1.58e-02	1.71e-02
COIL-20	AE	2.012	1.67e-02	-
	TopoAE	4.876	1.09e-02	-
	RTD-AE	16.404	1.90e-02	-
	NSA-AE	8.716	1.80e-02	-